Zellic

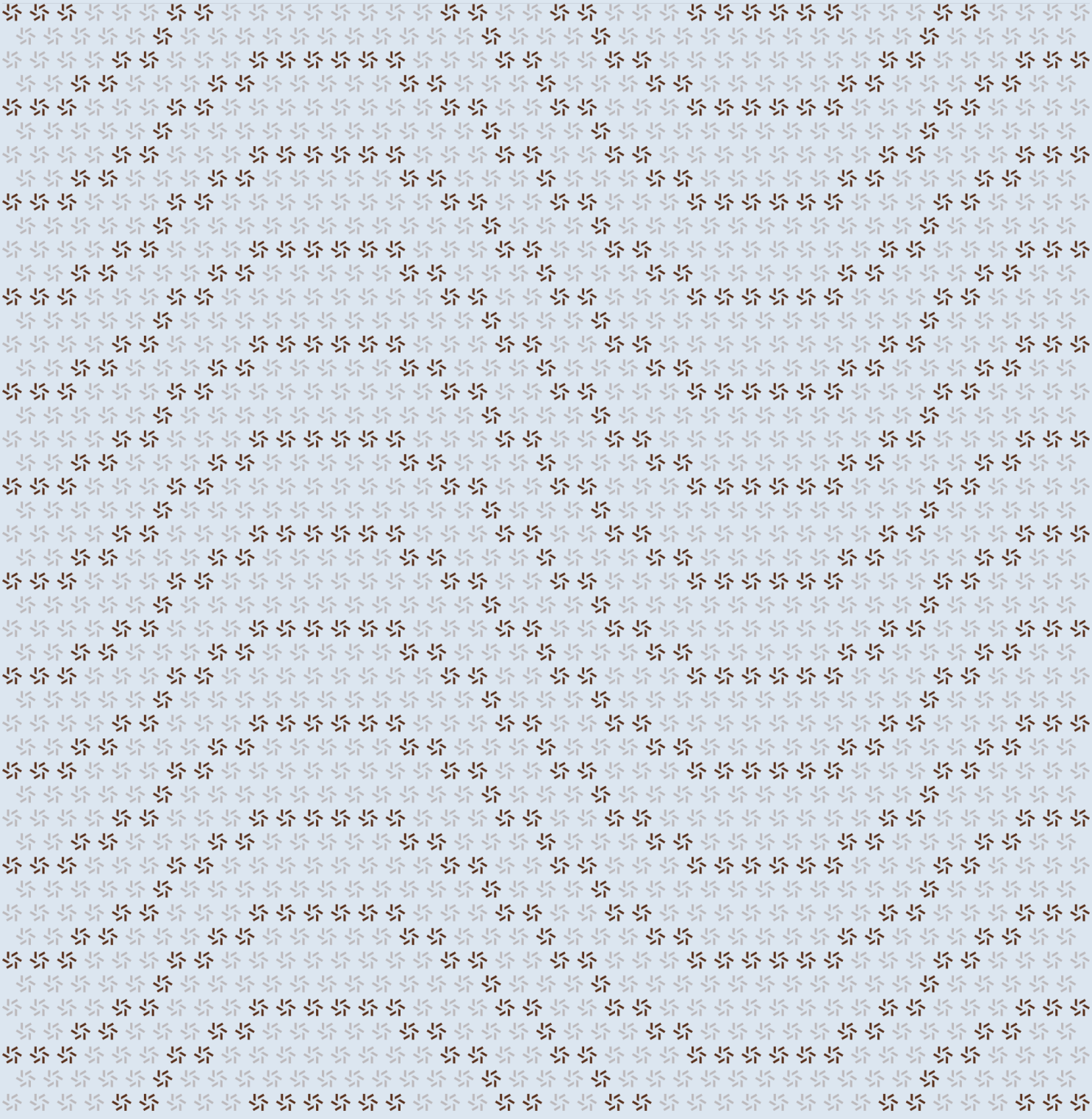
Prepared for
Standford Liu
Initia Labs

Prepared by
Sunwoo Hwang
Quentin Lemauf
Zellic

March 4, 2026

iUSD Vault

Move Application Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Initia Labs on March 3rd, 2026. During this engagement, Zellic reviewed iUSD Vault's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can an attacker infinitely mint iUSD?
- Can an attacker freeze the vault?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped iUSD Vault modules, we discovered one finding, which was of medium impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Initia Labs in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	0
Informational	0

2. Introduction

2.1. About iUSD Vault

Initia Labs contributed the following description of iUSD Vault:

> It is a relatively simple lock-and-mint contract. It was originally designed to support multiple collateral assets; however, we do not expect to require multi-collateral support in the near future.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the modules.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped modules itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

iUSD Vault Modules

Type	Move
Platform	Initia
Target	iUSD-vault
Repository	https://github.com/initia-labs/iUSD-vault ↗
Version	`eaa3b56c51609d95bc6327c24f754b257f57b523`
Programs	`sources/vault.move`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.5 person-days. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Sunwoo Hwang
Engineer
sunwoo@zellic.io ↗

Quentin Lemauf
Engineer
quentin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 3, 2026	Start of primary review period
March 3, 2026	End of primary review period

3. Detailed Findings

3.1. Fixed 1:1 mint/burn across collaterals enables cross-collateral arbitrage

Target	vault::vault		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

Mint and burn use the same raw unit amount across all registered collaterals. In `mint_with_fa`, the vault reads `amount` from the deposited token and mints exactly `amount` iUSD. In `burn_with_fa`, the vault burns `amount` iUSD and lets the caller withdraw exactly `amount` of any registered collateral they choose. If one collateral depegs, a user can mint with the weak asset and redeem a stronger asset at 1:1.

```
public fun mint_with_fa(
    base_token: FungibleAsset
): FungibleAsset acquires ModuleStore, VaultAccount {
    // ...
    let amount = fungible_asset::amount(&base_token);
    // ...
    let iusd = managed_coin::mint(&get_vault_signer(), get_iusd_metadata(),
    amount);
    // ...
    return iusd
}

public fun burn_with_fa(
    iusd: FungibleAsset, base_token_metadata: Object<Metadata>
): FungibleAsset acquires ModuleStore, VaultAccount {
    // ...
    let amount = fungible_asset::amount(&iusd);
    // ...
    managed_coin::burn(vault_signer, iusd_metadata, amount);
    let base_token = coin::withdraw(vault_signer, base_token_metadata, amount);
    // ...
    return base_token
}
```

Here is a textual proof of concept.

Token A and token B are both registered as base tokens. Market prices diverge; token A stays near peg while token B depegs. The vault holds redeemable token A liquidity from prior deposits.

1. The attacker deposits 10,000 token B through `mint` and receives 10,000 iUSD because minting is unit-based.
2. The attacker then calls `burn` for 10,000 iUSD while selecting token A as `base_token_metadata`.
3. Lastly, `burn_with_fa` checks only that token A is registered and that vault token A balance is sufficient, then withdraws 10,000 token A.

Impact

A depegged or weak collateral is converted into stronger collateral through iUSD at an artificial 1:1 rate. Healthy collateral reserves are drained, and resulting insolvency is socialized across the remaining users.

Recommendations

If the intended design is single-collateral, enforce that invariant on chain by preventing registration of a second base token.

If the intended design is multicollateral, implement oracle-based pricing for mint and burn so issuance and redemption are based on value, not raw token units.

Remediation

The Initia team acknowledged this issue and explained that the current implementation is intended to support only AUSD for the foreseeable future.

The team also stated that if additional collateral assets are introduced in the future, they plan to migrate mint and burn to an oracle-based pricing model to enforce value-based accounting and safer redemptions.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Code-quality improvements

These following notes on code quality are not directly security related.

Use two-step admin transfer for safer role handover

The function `update_admin` applies the new admin address immediately in a single transaction. This is simple, but it makes handover sensitive to operator mistakes such as typos, incorrect environment addresses, or transferring control to an account that cannot sign as expected.

```
public entry fun update_admin(chain: &signer, new_admin: address)
    acquires ModuleStore {
    check_permission(chain);
    let module_store = borrow_global_mut<ModuleStore>(@vault);
    module_store.admin = new_admin;
}
```

A two-step flow improves operational safety. The current admin sets `pending_admin`, and the pending address must call `accept_admin` to finalize the transfer. This pattern keeps ownership changes explicit and confirms the receiver controls the destination key before authority is switched.

Expand test coverage for invariants and adversarial paths

The current suite, as of the time of writing, covers core mint and burn behavior and several permission failures, which is a solid baseline. However, it does not fully exercise higher-risk invariant scenarios, especially around multicollateral state transitions and stress conditions. Additional tests for adversarial sequences and stateful edge cases would improve confidence in future refactors and parameter changes.

Consequences of limited depth in these paths include delayed detection of accounting regressions, weaker safety guarantees during upgrades, and higher risk that edge-case behavior is first discovered in production rather than in CI.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: vault

Description

The vault module manages the iUSD coin, minting iUSD against deposited base tokens and burning iUSD to redeem base tokens.

The vault module has the following responsibilities.

- Base-token management — registering base tokens (with optional `max_balance` caps) and updating `max_balance` per token
- Minting iUSD — accepting user base-token deposits, validating against registered tokens and `max_balance` limits, depositing base tokens into the vault object account, and minting iUSD 1:1 to the user
- Burning iUSD — accepting user iUSD; validating that the asset is iUSD, that the requested base token is registered, and that the vault has sufficient balance; burning iUSD; and releasing the requested base token to the user

Invariants

Access control

- Only the chain (@initia_std) or admin can call `register_base_token`, `update_max_balance`, and `update_admin`.
- Any account with base-token balance can call `mint`, and any account with iUSD balance can call `burn`.

Mint validity

- Base token must be registered.
- If the base token has `max_balance`, then `vault_balance(base_token) + amount <= max_balance`.

Burn validity

- The input asset must be iUSD.
- The requested base token must be registered.
- The user must have at least `amount` of iUSD.

Test coverage

Cases covered

- Registering a base token and rejecting duplicate registration
- Updating max balance for a registered token
- Rejecting register/update/admin when the signer is not the chain or admin
- Mint with registered token
- Mint exceeding max balance
- Mint with unregistered base token
- Burn with registered base token
- Burn when vault exceeds `max_balance`
- Burn with unregistered base token
- Burn exceeding vault balance of requested base token
- Burn with non-iUSD asset

Cases not covered

- Decimal mismatch between base tokens and iUSD
- Exchange between different base tokens
- Admin set to invalid or zero address

Attack surface

Entry points

The following functions are the public/entry functions that can be called to interact with the vault module.

- User-entry functions
  - `mint` — locks base token and mints iUSD to the signer
  - `mint_with_fa` — mints iUSD from a FungibleAsset
  - `burn` — burns iUSD and receives the specified base token
  - `burn_with_fa` — burns iUSD and returns a FungibleAsset
- Governance functions (chain or admin only)
  - `register_base_token` — registers a base token with optional `max_balance`
  - `update_max_balance` — changes `max_balance` for a registered base token
  - `update_admin` — changes the admin address

Mitigations

There are several mitigations.

- **Access control.** Governance functions enforce that the signer must be `@initia_std` or `module_store.admin`.

- **Registration checks.** The mint path uses `check_base_token_exists` and the burn path uses `check_base_token_exists`, so only registered base tokens are accepted.
- **Max balance.** The mint path uses `check_can_mint`, which enforces `balance + amount <= max_balance` when `max_balance` is set.
- **Burn liquidity.** The burn path uses `check_can_burn`, which ensures the vault holds at least `amount` of the requested base token.

6. Assessment Results

During our assessment on the scoped iUSD Vault modules, we discovered one finding, which was of medium impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.